

TECHNICAL DOCUMENTATION

Garmin Coach App

A local AI-powered running coach that visualises your training, and generates plans using the Claude API.

AUTHOR	DATE	VERSION
Nikanoras Papadopoulos	June 2026	1.0

Python · FastAPI

PostgreSQL

Claude AI

Playwright

Chart.js · HTMX

CONTENTS

1	What the system does	
2	Architecture overview	
3	Backend — FastAPI	
4	Database — PostgreSQL	
5	CSV Importer	
6	Playwright Sync (Browser Automation)	
7	Claude AI Integration	
8	Frontend — HTMX + Chart.js	
9	Data flow — end to end	
10	File reference	
11	Technology choices	

1 What the System Does

Garmin Coach is a **local web application** — it runs entirely on your own computer, not on any cloud server. You open it in a browser at `localhost:8000` , but all data stays on your machine.

In one sentence: it ingests your Garmin running data, stores it permanently, visualises your training history, and uses the Claude AI to generate personalised weekly plans and answer coaching questions.

Core capabilities

- **Data ingestion** — imports Garmin CSV exports (manual) or automates the export via a real Chrome browser (one-click sync)
- **Deduplication** — importing the same file twice never creates duplicate entries
- **Dashboard** — weekly mileage bar chart, pace trend, heart rate trend, personal records
- **AI weekly plans** — generates a structured 7-day training plan based on your actual stats and goals
- **AI coaching chat** — conversational interface with real-time context from your runs
- **Goals tracking** — CRUD interface for race targets and training goals

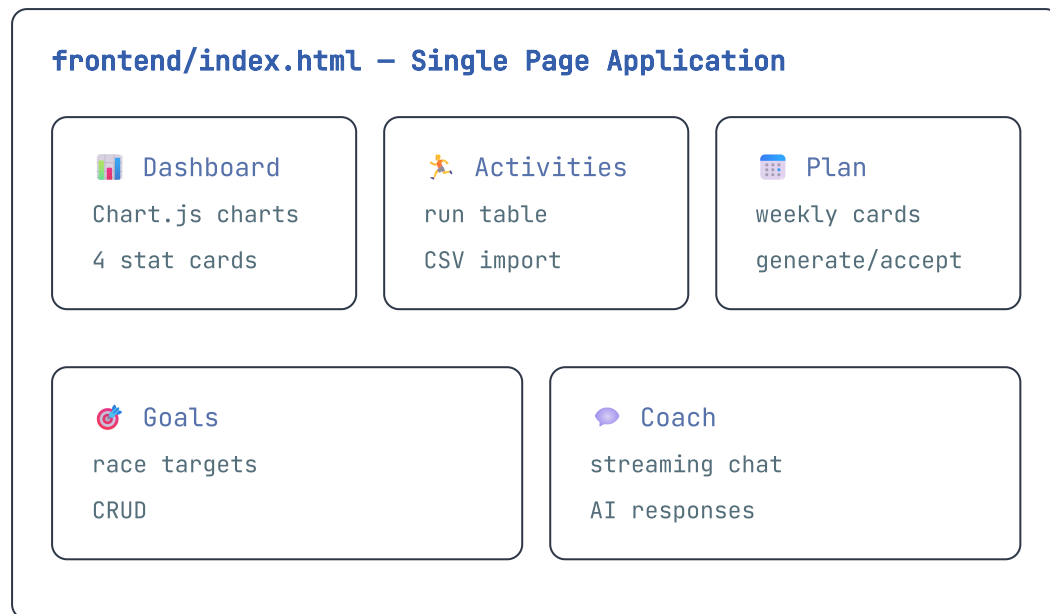


Why local? Garmin does not offer a public API for personal use. Running locally means no cloud fees, no third-party data sharing, and no rate limits. Your 859 runs and all AI responses stay on your machine.

2 Architecture Overview

The application has three layers: a **database** at the bottom for permanent storage, a **Python backend** in the middle that handles all logic, and a **browser UI** at the top for interaction. Two external services are called from the backend: Garmin Connect (via a controlled browser) and the Anthropic Claude API.

USER INTERFACE (BROWSER AT LOCALHOST:8000)



activities | goals |
weekly_plans

Anthropic
Claude API

claude-opus-
4-7
plans · chat

Garmin
Connect

Playwright
Chrome
CSV export
automation

3 Backend — FastAPI

FastAPI is the Python web framework that handles every HTTP request — from serving the HTML page to answering chart data queries to running the Garmin sync. It is chosen for its speed, automatic API documentation, and typed Python support.

The app is started with `uvicorn` (a lightweight async web server). The `--reload` flag makes the server restart automatically when any `.py` file changes — essential during development.

BACKEND/MAIN.PY — ENTRY POINT

```
from fastapi import FastAPI

app = FastAPI(title="Garmin Coach")

# Each router is a separate file of related endpoints.
# This is called "separation of concerns" – each file has one job.
app.include_router(activities_routes.router)
app.include_router(stats_routes.router)
app.include_router(plans_routes.router)
app.include_router(chat_routes.router)

@app.get("/", response_class=HTMLResponse)
def index():
    # Serve the single HTML file that is the entire UI
    return HTMLResponse(html_path.read_text())
```

Route modules

File	URL prefix	Responsibility
<code>routes/activities.py</code>	<code>/activities</code> , <code>/sync</code> , <code>/import/csv</code>	List runs, trigger Garmin browser sync, import CSV files
<code>routes/stats.py</code>	<code>/stats/*</code>	JSON data for all dashboard charts (weekly mileage, pace, HR, PRs)

<code>routes/plans.py</code>	<code>/plans/*</code>	Generate, accept, and archive AI weekly training plans
<code>routes/goals.py</code>	<code>/goals/*</code>	Create, read, update, delete race and training goals
<code>routes/chat.py</code>	<code>/chat</code>	Streaming AI coaching conversation with run context



Plain English: Think of each route file like a department in a company. The stats department only knows about charts. The activities department only knows about runs. They don't interfere with each other, which makes the code easy to maintain and extend.

4 Database — PostgreSQL

PostgreSQL is a production-grade relational database. We chose it over simpler alternatives (like SQLite or a plain JSON file) for two reasons: it handles concurrent reads/writes cleanly, and it supports **JSONB** — storing JSON objects inside a table column, which we use for the raw Garmin data and weekly plans.

Connection pooling

Instead of opening and closing a database connection on every HTTP request (slow), a **connection pool** keeps 2–10 connections permanently open and ready. When the app needs the database it borrows one, uses it, and returns it.

BACKEND/DB.PY — POOL SETUP

```
# min_size=2: always keep 2 connections open and ready.  
# max_size=10: never open more than 10 at once.  
_pool = ConnectionPool(  
    conninfo=config.DATABASE_URL,  
    min_size=2,  
    max_size=10  
)
```

The three tables

SQL SCHEMA (SIMPLIFIED)

```

-- Every run you have ever recorded.
CREATE TABLE activities (
  id          BIGINT PRIMARY KEY,
  start_local  TEXT,           -- "2026-06-04 07:32:00"
  start_minute TEXT UNIQUE,    -- deduplication key
  distance_km  NUMERIC,
  duration_min NUMERIC,
  avg_pace_min_per_km NUMERIC,
  avg_hr       INTEGER,
  vo2max       NUMERIC,
  raw_json     JSONB          -- full original data preserved
);

-- Claude-generated weekly training plans.
CREATE TABLE weekly_plans (
  id          BIGINT PRIMARY KEY,
  week_start  TEXT,
  plan_json   JSONB,          -- the full 7-day plan as JSON
  status      TEXT            -- 'proposed' / 'accepted' / 'archived'
);

-- Your race targets and training goals.
CREATE TABLE goals (
  id          BIGINT PRIMARY KEY,
  description TEXT,
  target_date TEXT,
  status      TEXT
);

```

Deduplication — the most important design decision

When you re-import the same CSV, or export a CSV that overlaps with a previous one, the database must not create duplicate entries. We solve this with an **upsert**:

THE UPSERT PATTERN

```

INSERT INTO activities (id, start_local, distance_km, ...)
VALUES (...)
ON CONFLICT (start_minute) DO UPDATE SET
  distance_km = COALESCE(excluded.distance_km, activities.distance_km),
  avg_hr      = COALESCE(excluded.avg_hr,      activities.avg_hr),
  ...

```

`start_minute` is the run's start time truncated to the minute (e.g. `"2026-06-04 07:32"`). Since two real runs can never start in the exact same minute, this is a reliable unique key. If a conflict is detected, `COALESCE` keeps whichever value is not null — so a richer earlier import (with HR data) is never overwritten by a sparser later one.



Result: You imported 13 CSVs containing 4,540 total rows across overlapping date ranges. The dedup system reduced those to 825 genuinely new runs — no duplicates, no data loss.

5 CSV Importer

Garmin exports CSVs in different formats depending on your account language. The importer auto-detects the locale and applies the right parser for each field.

Feature	English export	Greek export
Column delimiter	,	;
Decimal separator	4.5 km	4,5 km
Date format	2026-06-04 07:32:19	4/6/2026 7:32
Pace format	5:30 (MM:SS)	5:30:00 πμ (Excel time-of-day)
Headers	English text	Greek Unicode characters

BACKEND/CSV_IMPORTER.PY — LOCALE DETECTION

```
def _is_greek_locale(headers):
    # Check if any header contains a Greek Unicode character (range ι-Ϸ)
    return any(
        any("ι" ≤ ch ≤ "Ϸ" for ch in h) for h in headers
    )

def _parse_duration_greek(text):
    # Greek Garmin exports store pace as Excel time-of-day: "5:30:00 πμ"
    # πμ = AM, μμ = PM. We strip the suffix and read only MM:SS.
    text = text.replace(" πμ", "").replace(" μμ", "")
    parts = text.split(":")
    return int(parts[0]) + int(parts[1]) / 60 # decimal minutes
```

Synthetic IDs

Garmin's CSV format does not include a unique activity ID (unlike the API). We generate a deterministic ID by hashing the date + distance + duration. The same run always produces the same hash, making re-imports safe. IDs are made negative to distinguish them from real Garmin API IDs (which are positive).

DETERMINISTIC ID GENERATION

```
def _synthetic_id(date_str, distance_km, duration_min):  
    payload = f"{date_str}|{distance_km}|{duration_min}".encode()  
    h = hashlib.sha256(payload).hexdigest()[:15] # 60-bit hash  
    return -int(h, 16) # negative = CSV-sourced
```

6 Playwright Sync — Browser Automation

Playwright is a browser automation library from Microsoft. It can control Chrome, Firefox, or Edge programmatically — clicking buttons, filling forms, and capturing downloads — exactly as a human user would.



Why not the official Garmin API? Garmin's developer program is B2B-only — it requires a registered company and a formal application process. For personal use, browser automation against the same website you log into manually is the only stable path.

The persistent profile strategy

A fresh browser profile with no history, no cookies, and no saved preferences looks like a bot to Garmin's login system. We use a **persistent profile** stored in `data/chrome_profile/` that survives between runs. After the first login the session is reused automatically.

BACKEND/GARMIN_PLAYWRIGHT.PY — ANTI-DETECTION

```
# 1. Use the real Google Chrome installed on your machine (not a downloaded br
# 2. Point to a persistent profile directory that builds up history over time
# 3. Pass --disable-blink-features=AutomationControlled so Chrome does NOT
#    set the navigator.webdriver flag that Garmin checks
context = pw.chromium.launch_persistent_context(
    user_data_dir="data/chrome_profile",
    channel="chrome",
    headless=False,
    args=["--disable-blink-features=AutomationControlled"],
)

# 4. Inject JS into every page before it loads - overrides navigator.webdriver
context.add_init_script(
    "Object.defineProperty(navigator, 'webdriver', {get: () => undefined})"
)
```

The full sync flow

- 1 **Open Chrome → navigate to Garmin activities page**
`page.goto("connect.garmin.com/app/activities?activityType=running")`
- 2 **Wait for login (if session expired)**

Script pauses up to 5 minutes. You log in manually. After login, Garmin redirects to the home page.

3 Navigate back to activities page

Detects the post-login redirect and navigates back to the activities URL automatically.

4 Wait for "Εξαγωγή σε CSV" button (up to 45 seconds)

Garmin Connect is a heavy React app — it takes time to render. We poll for the button instead of assuming the page is ready.

5 Click export → capture download → save to `data/CSVs/garmin_sync_YYYY-MM-DD_HHMM.csv`

Playwright intercepts the file download before it hits the browser's Downloads folder and saves it directly to our data directory.

6 Auto-import the CSV → show count of new workouts

Counts activities before and after import. The difference is the number of genuinely new runs added.

WAITING FOR THE EXPORT BUTTON ROBUSTLY

```
def _wait_for_export_button(page, timeout=45_000):  
    # Try each locale label in turn  
    for label in ["Εξαγωγή σε CSV", "Export to CSV", ...]:  
        try:  
            loc = page.get_by_text(label, exact=True)  
            loc.first.wait_for(state="visible", timeout=per_label)  
            return loc.first  
        except PWTimeoutError:  
            continue  
    # Fallback: any button or link containing the text "CSV"  
    return page.locator("button, a").filter(has_text="CSV").first
```

7 Claude AI Integration

We use the **Anthropic Python SDK** to call Claude in two distinct ways: streaming chat for the coaching conversation, and structured JSON generation for weekly plans.

Streaming chat

The coaching chat uses **streaming** — Claude sends its response token by token as it generates it, rather than waiting until the full answer is ready. This makes the conversation feel live and responsive, like watching someone type.

BACKEND/ROUTES/CHAT.PY — STREAMING RESPONSE

```
with anthropic_client.messages.stream(  
    model="claude-opus-4-7",  
    system=f"""You are an expert running coach.  
The athlete's recent stats: {stats_summary}  
Their current goals: {goals}  
Their personal records: {personal_records}""",  
    messages=conversation_history,  
) as stream:  
    for text in stream.text_stream:  
        yield text    # each token sent to browser immediately
```



Context injection: Before calling Claude, we fetch your real run data from the database and embed it in the system prompt. Claude is not guessing — it is reasoning about your actual 859 runs, your specific goals, and your real personal records.

Weekly plan generation — structured output

For weekly plans we use a **structured prompt**: we instruct Claude to respond exclusively in a specific JSON format. We then extract the JSON with a regex, validate it has exactly 7 days, and store it in the database.

BACKEND/ROUTES/PLANS.PY — STRUCTURED JSON PROMPT


```
prompt = f"""
Generate a 7-day training plan. Respond ONLY with valid JSON,
no explanation, no markdown fences:
{{
  "week_theme": "...",
  "days": [
    {{
      "day": "Monday",
      "workout_type": "easy",    // easy/tempo/intervals/long/rest/cross
      "title": "Easy recovery run",
      "description": "...",
      "distance_km": 8,
      "duration_min": 50
    }},
    ...    // 7 days total
  ]
}}
```

Athlete context:

- Total runs: {stats["total_runs"]}, Total km: {stats["total_km"]}
- This week: {stats["this_week_km"]} km
- Goals: {goals_text}

```
"""

# Extract JSON from response using regex (robust to extra text)
match = re.search(r'\{.*\}', response, re.DOTALL)
plan = json.loads(match.group())
```

Workout badge colours

Type	Colour	What it means
Easy	green	Low heart rate, conversational pace, recovery
Tempo	orange	Comfortably hard, lactate threshold training
Intervals	red	High intensity repeats with recovery between
Long	blue	Weekly long run at easy pace, builds endurance base
Rest	grey	Full rest or very light activity
Cross	purple	Cross-training (cycling, swimming, gym)

8 Frontend — HTMX + Chart.js

The entire frontend is a **single HTML file** — no React, no Vue, no `npm install`, no build pipeline. For a personal local tool, the simplest approach that works is the correct one.

HTMX — server-rendered interactivity without JavaScript

HTMX is a small library (~14 KB) that lets HTML elements make HTTP requests and swap the server's response into the page — without writing any JavaScript. The server builds the HTML; HTMX handles the swap.

EXAMPLE: THE SYNC BUTTON

```
←!— When clicked: POST to /sync, show spinner, put response into #activities-
<button
  hx-post="/sync"
  hx-target="#activities-panel"
  hx-indicator="this">
  Sync from Garmin
</button>

←!— This div auto-loads its content from /activities on page load →
<div id="activities-panel"
  hx-get="/activities?days=30"
  hx-trigger="load">
  Loading...
</div>
```

Chart.js — the dashboard charts

The four dashboard charts use Chart.js 4.4.3, loaded from a CDN. The browser fetches JSON from five `/stats/*` endpoints simultaneously using `Promise.all()`, then renders all charts at once. Charts only initialise when the Dashboard tab is first opened — not on every page load.

LAZY CHART INITIALISATION

```
let dashReady = false;

function showTab(id) {
  // Only fetch + render charts the first time Dashboard is opened
  if (id === 'dashboard' && !dashReady) {
    dashReady = true;
    initCharts();
  }
}

async function initCharts() {
  // Fetch all 5 endpoints in parallel
  const [summary, mileage, pace, hr, prs] = await Promise.all([
    fetch('/stats/summary').then(r => r.json()),
    fetch('/stats/weekly-mileage').then(r => r.json()),
    fetch('/stats/pace-trend').then(r => r.json()),
    fetch('/stats/hr-trend').then(r => r.json()),
    fetch('/stats/prs').then(r => r.json()),
  ]);
  // Render charts with the received data...
}
```

Responsive layout

Screen size	Navigation	Charts
Desktop (> 767px)	Horizontal tab bar at top	2 × 2 grid
Mobile (< 767px)	Fixed bottom nav with emoji icons 📅 🏃 🗣️	Single column stack

No CSS framework is used. All styling is custom CSS using variables for the dark theme, making it trivial to change the colour scheme in one place.

CSS CUSTOM PROPERTIES (THEME VARIABLES)

```
:root {  
  --bg:      #0f1117;    /* page background */  
  --panel:   #1a1d27;    /* cards and tables */  
  --accent:  #4f8cff;    /* buttons, links, highlights */  
  --border:  #2d3748;    /* dividers */  
  --muted:   #98a2b3;    /* secondary text */  
}
```

9 Data Flow — End to End

When you click "Sync from Garmin"

Browser → `POST /sync`

→ `garmin_playwright.export_csv()`

→ Chrome opens with saved session (data/chrome_profile/)

→ navigate to `connect.garmin.com/app/activities?`

`activityType=running`

→ `if not logged in`: wait up to 5 min for you to log in

→ wait up to 45s for "Εξαγωγή σε CSV" button to appear

→ click button → download captured → saved as `garmin_sync_2026-`

`06-07_1430.csv`

→ `csv_importer.import_csv_file(path)`

→ detect locale (Greek or English)

→ parse rows → generate synthetic IDs → upsert into PostgreSQL

→ `count_after - count_before = new workouts added`

→ return HTML banner + refreshed table

Browser ← "Added 3 new workouts from Garmin" + updated table

When you open the Dashboard

Browser → `fetch('/stats/summary')` → 4 totals for stat cards

→ `fetch('/stats/weekly-mileage')` → bar chart labels + values

→ `fetch('/stats/pace-trend')` → line chart dates + paces

→ `fetch('/stats/hr-trend')` → HR line chart data + paces

→ `fetch('/stats/prs')` → fastest 5k / 10k / HM /

marathon

(all 5 run in parallel)

Chart.js renders all charts simultaneously

When you generate a Weekly Plan

Browser → `POST /plans/generate`

→ fetch stats + recent runs + goals from PostgreSQL

→ Anthropic Claude API (claude-opus-4-7)

system: "You are an expert running coach. Athlete stats: ..."

user: "Generate a 7-day plan as JSON"

← Claude returns JSON with 7 workout days

→ validate JSON has exactly 7 days

→ store in weekly_plans table (status='proposed')

→ render as HTML day-cards

Browser ← 7 coloured workout cards with Accept / Regenerate /

Archive buttons

10 File Reference

```

garmin-coach/
├── backend/
│   ├── main.py           # FastAPI app entry point – mounts all
│   │   routers
│   ├── config.py         # loads .env, exposes DATABASE_URL, API
│   │   keys, paths
│   ├── db.py             # ALL database functions: schema, upsert,
│   │   queries
│   ├── csv_importer.py   # multi-locale Garmin CSV parser +
│   │   importer
│   ├── garmin_playwright.py # browser automation sync via
│   │   Chrome/Playwright
│   ├── sync.py           # legacy API sync (kept but unused)
│   └── routes/
│       ├── activities.py  # /activities /sync /import/csv
│       ├── stats.py       # /stats/* – JSON for all dashboard charts
│       ├── plans.py       # /plans/* – AI weekly plan generation
│       ├── goals.py       # /goals/* – CRUD for race targets
│       └── chat.py        # /chat – streaming AI coaching
│   conversation
├── frontend/
│   └── index.html         # entire UI – all CSS, HTML, JS in one
│   file
├── scripts/
│   └── bulk_import_csvs.py # imports many CSV/xlsx files at once
├── data/                  # ← gitignored: never committed
│   ├── CSVs/              # raw Garmin CSV exports
│   └── chrome_profile/     # Playwright persistent browser session +
│   cookies
├── requirements.txt       # Python dependencies
├── start_app.bat          # double-click launcher for Windows
└── .env                   # secrets: API key, DB password – never
in git

```

11 Technology Choices — The "Why"

Technology	Role	Why we chose it
FastAPI	Web framework	Modern Python, async-capable, auto-generates OpenAPI docs, typed — faster to develop than Flask/Django for API-first apps
PostgreSQL	Database	JSONB support for raw Garmin data, proper upsert with conflict resolution, connection pooling — production-grade even for a personal tool
psycopg3	DB driver	Newest Postgres Python driver; native async support and built-in connection pooling in a single package
Playwright	Browser automation	Controls a real browser — immune to API changes, handles JavaScript-heavy SPAs, supports persistent profiles for session reuse
HTMX	Frontend interactivity	Server-rendered HTML without a JS framework — no build step, no node_modules, simpler mental model for a personal tool
Chart.js	Data visualisation	Well-documented, CDN-ready (no install), handles all chart types needed with good default styling
Claude API	AI engine	Best reasoning for coaching context; streaming API makes chat feel live; structured JSON output for plan generation is reliable
Single HTML file	Frontend	No build pipeline, no transpilation, no dependency management — open it in any browser and it works



Scale as of June 2026: 859 runs · 3,949 km of data spanning May 2024 – June 2026 (25 months). Imported from 13 CSVs across multiple overlapping date ranges. Zero duplicates.